

AMAZON HACKATHON · RETURN PIPELINE

ReLoop

The intelligence layer for Amazon's returns pipeline.

Grade at the doorstep — decide the item's best next life **before it moves.**

“The landfill is a design flaw.”

Deterministic glass-box routing engine · our own DINOv2 grader · Product Health Card · live at reloop-woad.vercel.app

Amazon already resells returns — but decides at the worst moment

Renewed · Resale · Grade-and-Resell all grade **AFTER** the linehaul. Every leg is spent, weeks of value decay — **then** anyone decides what the item is worth.

Leg (today's journey)	Cost / dwell accrued	ReLoop's effect
1 Click Return → label issued	—	Grades HERE, routes before anything moves
2 Pickup → delivery station	Last-mile (spent either way)	Driver checkpoint rides along free; local items STOP here
3 Station → sortation center	First avoidable leg	DELETED for local routes
4 Sortation → RLC linehaul	The big 580 km freight leg	DELETED for local routes
5 RLC queue → inspection	Days-to-weeks of dwell = decay	Replaced by a 10-min hub bench in-city
6 Manual grade → disposition	Decision made LAST, costs sunk	Made FIRST (stage 0–3), re-checked at checkpoints
7 Execute (often a 2nd linehaul)	Another leg	Restock → nearest FC; pallets batch at the hub

Net today: 5–7 touches · 2–6 weeks · only **10–20% ever restocked**. Every leg from #3 on is avoidable — if the decision exists before leg #3.

Information-timing arbitrage

Information only has value if it changes a decision before money is spent.

Use **noisier information (doorstep photos)** at the moment of maximum leverage, instead of **perfect information (physical inspection)** after every cost is already sunk. Inspection still happens — it just moves to the cheapest node that precedes irreversibility.

WHY RETURN, NOT TRADE-IN

- ▶ **Trade-In:** destination is fixed (Amazon's facility) whatever the grade — doorstep AI changes nothing physical. Pure cost.
- ▶ **Return:** the destination IS the decision. Doorstep AI changes where the item physically goes.

BOUNDED DOWNSIDE BY DESIGN

- ▶ Every wrong local decision degrades to “send it up the chain” — what happens to 100% of items today.
- ▶ **The bar:** $P(\text{correct}) \cdot \text{savings} > P(\text{wrong}) \cdot \text{correction}$. Correction = a shelf move at a hub, not a lost item.

Grade at the source. Decide before the item moves.

Amazon's current workflow is the engine's **fallback**, not its competitor. We are the missing front end of programs Amazon already runs.

Decide FIRST

The route is chosen at stage 0–3 — at the doorstep — then re-checked at two physical checkpoints while redirect is still cheap.

Delete the legs

Locally-routed items stop at the in-city delivery station. Zero new transport legs added; sortation + 580 km linehaul deleted.

Glass-box engine

One deterministic expected-value optimizer. Logic decides, the LLM only narrates. Every destroy has a replayable reason.

Four pillars · ML for perception, rules for the decision

AI Grading

“the eyes”

Our own DINOv2 grader → a grade DISTRIBUTION + defect tags. VLM narrates defects; OCR anchors identity.

Smart Routing

“the brain”

Deterministic, explainable EV engine over {value, cost, demand, carbon}. Glass-box on purpose.

Health Card

“the trust”

Verifiable condition, history & authenticity that travels with the item to its next owner. CARFAX for a product.

Prevention

“the upstream”

Predict returns before they happen — the cheapest return is the one that never occurs.

THE STACK — pnpm monorepo, strict TypeScript, no any

apps/web

Next.js App Router + Tailwind — Return flow, Hub Bench, Shop, seller dashboard

apps/api

Node + Express — grading, /api/route + /api/return/checkpoint, pricing, matching cron

packages/shared

Single source of truth for data contracts: routing-ev.ts, liquidation-lot.ts, return.ts

ml/

DINOv2 grader (perception) · XGBoost + Thompson-bandit pricing (spec 014) · return-risk

One expected-value optimization, per returned item

$$\begin{aligned} \text{EV}(r) = & \sum_g P(g|\text{evidence}) \cdot \text{recovery}(r, g) \cdot \text{decay}(t_r) \\ & - \text{logistics}(r) - \text{handling}(r) \\ & - \mathbf{E[\text{correction_cost}(r)]} - \lambda \cdot \text{CO}_2(r) \end{aligned}$$

Choose $r \in \{\text{restock}, \text{local_resale}, \text{refurbish}, \text{liquidate}, \text{donate}, \text{recycle}, \text{warehouse}, \text{return_to_seller}, \text{returnless_refund}\}$

THREE TERMS ARE THE UPGRADE OVER A NAÏVE ROUTER

$P(g|\text{evidence})$ — posterior, not a point grade

Routes differ in error-sensitivity. Restock is brutal on a wrong A; donation barely cares. Recovery is an expectation over the grade distribution.

$\text{decay}(t_r)$ — time is a P&L line

Each route has an expected time-to-cash; category-specific weekly decay makes the engine SEE that weeks of returns-center dwell burn money.

$\mathbf{E[\text{correction_cost}(r)]}$ — being wrong has a price

= (posterior mass below the grade the route needs) $\times$ that route's redirect cost. This is what makes the confidence gates θ_r derived, not hand-picked.

The hard-constraint ladder (safety · legal · policy)

An ordered, first-match ladder evaluated **before** any EV math. It can force a path and is audited for 100% conformance every build.

1	3P seller not opted in	→ return_to_seller
2	counterfeit / not-as-described	→ return_to_seller (policy)
3	hazmat / restricted	→ recycle (certified disposal only)
4	wrong item	→ warehouse (inventory reconciliation)
5	authenticity mismatch	→ warehouse (manual verification)
6	high-value + unverified	→ warehouse (fraud/verification gate)
7	reason ↔ grade mismatch	→ warehouse (fraud review)
8	salvage / ungradeable	→ recycle
9	arrived damaged	→ recycle

 **PASTE SCREENSHOT HERE**

Run: `pnpm eval`

Routing hard-rule conformance — forced-path
accuracy 100.0% (N=8)

Determinism is a business requirement

Every destroy and every denied local route needs a replayable reason for seller disputes and ESG audits. The ladder is that record.

Every path priced, every term signed — glass-box

Sealed changed-mind electronics. The engine prices all 8 paths and picks the max.
Restock wins — straight to the nearest FC, deleting the returns-center hop.

```
POST /api/route → evBreakdown

decision: restock      (localMargin ₹1,952)

restock      ₹1,956 ✓ CHOSEN
  Restock at full recovery      +₹2,299
  Grade uncertainty discount    -₹21
  Value decay (4d to cash)     -₹26
  FC inbound 45km              -₹90
  Receive + shelve             -₹150
  Expected correction cost     -₹55
local_resale  ₹1,952
refurbish    ₹1,733 (not viable)
liquidate    ₹893
donate       ₹182
warehouse   -₹465 ← today's flow
```



PASTE SCREENSHOT HERE

Run: `curl /api/route` (Scenario A)

The full evBreakdown JSON — every path's signed EV terms. This is the glass box.

Warehouse is -₹465 — the honest 580 km freight makes today's default the worst commercial option. Every better route is pure margin.

Confidence gates θ_r — graceful degradation to today's flow

θ_r is **derived from correction cost**, not arbitrary — its ordering mirrors redirect-cost ordering exactly. Low confidence collapses the eligible set toward the cheapest-to-correct path.

Route	θ_r	Redirect cost (why)
restock	0.85	second return + trust (~₹700 on a ₹2.5k item)
local_resale	0.60	hub shelf move (₹40)
refurbish	0.50	wasted bench slot (₹80, pre-buyer)
donate	0.30	being wrong is nearly free
liquidate	0.20	pallet re-sort (₹15) — cheapest commercial gate
recycle / warehouse	ungated	the absolute fallback — always available

gateReason is returned per path, so the UI shows a path's full EV next to WHY it was gated — the glass box never hides the road not taken.

 **PASTE SCREENSHOT HERE**

Run: `curl /api/route` (Scenario B)

*confidence 0.35 → local_resale & refurbish GATED OUT;
collapses to the liquidate pallet.*

The money line

A wrong grade on a manifested pallet costs ~₹15 to re-sort — vs a 580 km round trip. Low confidence collapses toward the **cheapest correction, not the most expensive one.**

A return is a lifecycle, not one decision

The engine re-runs at every physical checkpoint — information improves and redirect cost rises as the item moves.



EXECUTION (each still re-enters the engine on failure):



EVERY TRANSITION RE-INVOKES THE ENGINE

“Demand changed while routing was planned” stops being an edge case — it's just a re-evaluation at the next checkpoint. Static fallback chains are replaced by “re-run the engine from the current state.”

EVERY STATE HAS A COST-TO-REDIRECT

At ROUTED it's zero. At AT_LOCAL_HUB it's a shelf move. At SOLD it's a buyer notification. The engine reasons about **commitment**, not just value — and a decision TTL forces re-evaluation on expiry.

Two human checkpoints keep the AI honest — live re-route

Driver scan (30s at the door) + hub bench (10 min in-city). When the human disagrees with the AI, the **same engine re-runs instantly** — the wrong grade is caught before any buyer is exposed. Correction cost: one shelf move.

```
● ● ● live re-route (Scenario D)

BEFORE — doorstep, sealed grade A
POST /api/route
→ decision: restock    ttlHours: 24

AFTER — hub bench: observedGrade B,
        seal broken → engine re-runs
POST /api/return/checkpoint
→ decision: local_resale
→ 'Hub bench overrode grade A → B;
   engine re-ran and routed to
   local_resale.'
→ transition: at_local_hub →
   hub_verified
```

 **PASTE SCREENSHOT HERE**

Run: Hub Bench → toggle seal

The decision flips restock → local_resale on screen as you break the seal.

THE FLYWHEEL — the data moat

Every bench verdict is a free labelled training pair (photos → verified grade). Our DINOv2 grader improves as a **byproduct of operations** — graded pallets and audited destruction are second-order wins nobody gets without grading at source.

Where the money is — eight routes, one engine

Route	Trigger	vs Amazon today	Saved / earned
Restock	Sealed / verified-A + benign reason + SKU live	Rides to RLC, dwells weeks, maybe restocks	Straight to nearest FC; deletes hop + dwell
Local resale	Grade A/B, mid-value, demand clears	Liquidated for cents or resold weeks later	60–80% of clearing; zero linehaul; cash in days
Refurbish	Defect with positive uplift	G&R grades at FC, no repair at source	₹300 cable → B→A worth +₹1,500; re-enters resale
Liquidate	Functional, low unit EV, or high uncertainty	Residual after items sit at the RLC	Manifested pallets → top of the band; FC hop gone
Donate	All commercial EVs < donation credit	Same, after full RL cost sunk	Half handling, zero freight, CSR credit
Recycle	Forced (hazmat) or materials > handling	Same, after two truck rides	Local certified drop; commodity + carbon credit
Destroy	Legally required only	A real % die by default	Engine KPI is SHRINKING this — logged reason
Standard RL	Low conf / fraud / opt-out (fallback)	Identical to today	Zero, by design — the graceful-degradation guarantee

We priced the engine against real Amazon returns economics

Every constant is grounded in how Amazon's own machine actually prices things — no fictions.

FBA Liquidations nets sellers 5–10% of ASP

paid 30–90 days out; Amazon sells to liquidators at 20–30¢ on the retail dollar.

~\$27 to process a \$100 return

restock + shipping + inspection labor — for cheap items this exceeds every recovery path.

Returns fraud runs 9–15%

so any refund-without-return lever must hard-gate on trust + fraud, never on value alone.

Manifested pallets clear materially higher

than mystery lots — the exact premium a Health-Card manifest earns.

WHAT CHANGED IN THE ENGINE

- ▶ **Warehouse repriced:** the flat 60%-recovery fiction is gone. Now a mixture — 15%×85% restock-after-inspection + 85%×20% FC liquidation, then 21-day dwell decay.
- ▶ **liquidate is first-class:** a Health-Card-manifested hub pallet with its own lot engine (next slide).
- ▶ **E[correction_cost] built:** the EV term that was documented but never implemented — now real and visible.
- ▶ **Defect-level refurb:** a DEFECT_REPAIR_TABLE (tag → repair ₹ + grade delta) replaces grade-level fractions.
- ▶ **returnless_refund:** when every movement path loses money and trust/fraud/value gates pass — refund, keep the item.

Manifested pallets: a real secondary-market auction

Deterministic, glass-box like routing. A pallet = a grade histogram + a manifest. Per-buyer bid curves (refurbisher, wholesaler, NGO, fashion) auction it; the Health-Card manifest premium is what grading-at-source earns. A closed-form ship-vs-wait breakeven decides pickup timing.

```
tsx src/scripts/lot-trace.ts

Pallet: 30 electronics · avg ₹2,400 · 90% manifest
Winning buyer: refurbisher
  gross          ₹31,877
  Amazon take 10% ₹3,188
  seller proceeds ₹28,689
  14× grade B @ 34% +₹11,424
  Manifest premium +₹9,893
2nd-best (auto re-match): wholesaler ₹23,803

Manifested (90%): ₹28,689
Mystery lot (0%): ₹19,786
Premium earned: ₹8,904 (+45%)

ship-vs-wait n* = ceil(sqrt(F·λ/(δ·v)))
40/40 → shipNow=true (pallet full)
```

 **PASTE SCREENSHOT HERE**

Run: `tsx src/scripts/lot-trace.ts`

The pallet buyer auction + the +45% manifest premium + ship-vs-wait verdict.

WHY THIS IS A MOAT

- ▶ A returns center can't reprice a queue. We turn a mystery lot into a graded, transparent pallet a buyer can inspect.
- ▶ **+45% recovery** on the same physical units, purely from the manifest the doorstep grade already produced.

Returnless refund — hard-gated, deterministic

Amazon's real lever, decided by math instead of ad hoc. When **every movement path has negative EV** and all gates pass, refund and let the customer keep the item — zero legs, zero handling, zero carbon.

Value gate

never for items $\geq ₹800$ — high value must be physically recovered.

Fraud gate

any wardrobing / photo-reuse / auth-mismatch signal hard-blocks it.

Trust gate

$\text{customerTrust} \times \text{confidence}$ must clear 0.5 (the pickup-refund lever).

All-paths-negative

excluded from the argmax; fires ONLY when even the best real route loses money.

Opt-in via **customerTrust** — existing demo flows are byte-unaffected unless a caller supplies it.



PASTE SCREENSHOT HERE

Run: `pnpm test:edge`

all-paths-negative + trust $\rightarrow$ returnless_refund

THREE NEGATIVE CONTROLS PASS

- ▶ no trust signal $\rightarrow$ ineligible, stays a real route
- ▶ high value $\rightarrow$ must be physically recovered
- ▶ fraud signal $\rightarrow$ blocked

“Listed locally” is a state that still has to be won

- ▶ **The Listing Agent** (spec 008) is born the moment the hub bench confirms local_resale: it mints a Health Card from the return's own checkpoints and spawns on a real marketplace listing.
- ▶ **Floor = route-elsewhere EV.** The agent's hard price floor is seeded from max(warehouse, liquidate) EV — it escalates exactly when local resale stops beating “send it up the chain.” Spec-016 economics drive spec-014 escalation with zero new logic.
- ▶ **Event-driven repricing (spec 014).** Significant events (comp undercut, view-velocity drop, dwell) hit an XGBoost reward model + Thompson bandit + deterministic guardrails. A returns center cannot reprice a queue — this is the unique capability.
- ▶ **Escalation re-enters the Bridge.** listed_local → donation_batch | recycle_batch through the same state machine. Items never rot on a shelf.

THE DEMAND GRAPH — additive on Amazon's rec system

Amazon already knows who searched, wishlisted, or bought similar near the hub. We score that intent instead of inventing a marketplace:

```
matchScore = intentWeight  
             (searched .65 / wishlist .8 / bought .9)  
             × distanceDecay × priceFit
```

Buyer surfaces: the exact product page shows “Open-box near you · doorstep graded · hub verified · X% off · delivered today,” with the recommendation layer speaking (“this is on your wish list”). The purchase runs a real cross-account transaction that closes the lifecycle: listed_local → sold → delivered_to_buyer, EcoCredits in both ledgers, agent retires.

Stateful, restart-safe buyer matching — a cron cascade

No fragile `setTimeout` chains. Timeouts are detected from stored timestamps, so match state survives a server restart. One node-cron pass every 30 minutes.

`handleTimeouts()`

a notified buyer who didn't respond within the 2-hour window → mark timeout, cascade to the next candidate. Runs off `notified_at`, never a live timer.

`retrySearches()`

sessions with no candidates left → re-run `findCandidates` in case new buyers registered; throttled to once / 2h via `updated_at`.

`handleExpiry()`

pickup window closed without a match → `status = warehouse_fallback`, item's `local_routing_accepted = false`. The item never waits forever.

```
structured cron logs
{"level":"info","msg":"matching cascade scheduled",
  "cron":"*/30 * * * *"}
{"level":"warn","msg":"match session expired – falling back to warehouse","returnId":...}
{"level":"info","msg":"matching cascade complete",
  "timeoutsAdvanced":1,
  "searchesRetried":2,
  "candidatesFound":1,
  "sessionsExpired":0}
```

Every cron pass emits a summary line + a request id per API call — greppable now, ingestible into CloudWatch/Datadog later with no code change.

Every edge case maps to an owning mechanism

Edge case	Owning mechanism
Package opened / packaging missing	Normal case — completeness vector + seal check move the posterior; hub repack is the default workflow
Accessories / cables missing	VLM manifest check → refurb (uplift positive) or grade-down
Low AI confidence	θ_r gates shrink the eligible set toward liquidate/warehouse — no special-case path
AI grade wrong	Caught at driver scan / hub bench before buyer exposure; bounded correction; verdict becomes a training label
Customer hides damage	Driver checkpoint + refund timed to $\text{trust} \times \text{confidence}$ + photo-reuse / capture attestation
Local buyer rejects	Item still in-city; re-enters at hub; one local trip
No nearby buyer	Demand curve priced it → engine chose another route; unsold listing → dwell expiry → re-run
Demand shifts mid-route	Decision TTL; re-evaluated at each checkpoint while redirect is cheap
Unsellable after inspection	Hub cascade: pallet → donate → recycle, same building
Special-handling category	Hard ladder, evaluated first, never EV-optimized

Honest scope — what's real, what's mocked, what's next

PRODUCTION-SHAPED TODAY

- ▶ **Deterministic + pure** core engine (routing-ev, liquidation-lot) — no randomness, replayable, unit-tested. Scales as pure compute.
- ▶ **Structured JSON logging** one line/event with a reqId per request (X-Request-Id echoed) — CloudWatch/Datadog-ready.
- ▶ **Hardening** helmet security headers, per-IP rate limiting (300/min), bcrypt auth, input validation on every route.
- ▶ **Restart-safe jobs** cron cascade + demand rollup detect state from timestamps, not in-memory timers.
- ▶ **Drift watchdog** PSI monitor → auto-fallback when input distribution shifts (proven in eval).

SCOPED / NEXT (stated plainly)

- ▶ **Per-item greedy + batch-at-hub** is the honest scope. Fleet-level assignment (pickup batching, pallet consolidation, hub capacity) is a real production concern — documented, deliberately not built.
- ▶ **Mocked for reproducibility:** SKU-prefix economic profile, per-category demand & decay params, secondary-market bid curves — same posture as the existing pricing mock.
- ▶ **Real:** grading/pricing model calls, the full EV + lot engines, the state machine, checkpoint re-evaluation, MongoDB persistence, the reprice bandit.

Reproducible proof — no network, no key, one command each

51 / 51

edge-case matrix passes

`pnpm test:edge`

100%

routing hard-rule conformance

`pnpm eval`

100%

argmax-EV optimality

`pnpm eval`

5.3%

pricing MAPE (₹186 MAE)

`pnpm eval`

0.771

return-risk AUC vs 0.715 prior

`pnpm eval`

0.099→0.024

calibration ECE (temp-scaled)

`pnpm eval`



PASTE SCREENSHOT HERE

Run: `pnpm eval && pnpm test:edge`

The full deterministic eval report + the 51/51 edge-case matrix — the baseline every later ML phase must beat.

Live, auto-deployed, wired end-to-end

Web — Vercel

`reloop-woad.vercel.app`

Next.js (apps/web). NEXT_PUBLIC_API_BASE_URL baked in at build.

API — Render

`reloop-api-po73.onrender.com`

Express (apps/api) via tsx. NVIDIA_API_KEY + WEB_ORIGIN (CORS).

Data — MongoDB

`cloud state sync`

Per-user + shared state, returns, match sessions, pricing indexes.

Both auto-deploy from `main`. Pre-push gate: `pnpm -r typecheck` (strict, no any) + `web build`. @reloop/shared ships as TS source — API runs raw via tsx.

Amazon built the resale rails.

ReLoop gives them eyes at the doorstep
and a brain that decides before the truck rolls.

The return pipeline, rewired — the Amazon way.

reloop-woad.vercel.app

· *“The landfill is a design flaw.”*